
Two Coding Agents Optimize GraphViz: A Controlled Comparison

Ryan Tran¹, Hailu Xu¹

¹California State University, Long Beach
hailu.xu@csulb.edu

Abstract

Can a coding agent find and fix a real performance bottleneck on its own, with no hint about where the program spends its time? We gave two agents — Claude Code and Codex — the same untouched copy of GraphViz and the same instruction: make the `dot` layout program faster without changing what it draws. Each agent ran four times, eight runs in all, and no run could see the benchmark we would judge it on. Half the runs got an extra instruction to use the machine’s idle cores.

Every run found the same hot code. What differed was how far each went. All four Claude runs improved three of the five layout phases, making layout $1.39\times$ to $2.91\times$ faster. Of the four Codex runs, one improved a single phase ($1.03\times$ to $1.33\times$) and three shipped no code at all — each built something, measured it, found it slower or not worth keeping, and reverted. We report those as decisions rather than failures, and they left us with three binaries that were the unmodified program under another name, which is a useful check on our own noise estimates.

Two results surprised us. Repeating a prompt changed the outcome more than changing the agent did: Claude’s two identical runs differ by $1.39\times$ against $1.67\times$ on one graph, and Codex’s differ in whether anything shipped. And asking for concurrency is not the same as getting it — two Claude runs look equally parallel end to end, but re-running each on a single thread shows threads account for $2.32\times$ of one and 16% of the other. How much a phase is worth depends on the shape of the graph, not its size: crossing minimization is 12.4% of layout time on one shape and 64.8% on another with the same number of nodes. We measured how far apart two runs of the *unmodified* program land, and report only differences larger than that.

1 Introduction

Coding agents are now routinely asked to modify real software. Most evaluations of that ability measure whether an agent can make a failing test pass [6] — a task with a clear oracle, where success is checkable by running the suite. Performance optimization has no such oracle. An agent asked to make a program faster must first decide *what* to make faster, and a plausible-looking change that helps nothing is indistinguishable from a real improvement unless someone measures carefully.

This paper asks a narrow version of that question. Given a mature C codebase, no hint about where time is spent, and an instruction only to make the program faster without changing its output, can a coding agent find a genuine bottleneck and fix it?

We use the `dot` layout engine from GraphViz [3]. It is real, widely deployed software rather than a benchmark written for the occasion; its layered pipeline [8, 5] has distinct phases whose costs can be measured separately, so a speedup can be attributed rather than merely observed; and its text output is deterministic, which makes “the output did not change” checkable rather than a judgment call.

What we did. Two agents — Claude Code and Codex — each ran four times, for eight runs in total. Each run received its own pristine checkout of GraphViz pinned to commit 23b06521 and a prompt naming no target function, file, or phase. Two runs per agent used a base prompt; the other two added a section asking the agent to treat concurrency as its primary strategy, because no run under the base prompt had introduced any. Every prompt forbade the shortcuts that would fake a win: caching layout results, special-casing inputs, and reducing the iteration counts that govern layout quality. No run had access to the evaluation harness, the graphs it uses, or any other run’s sandbox. We rebuilt all eight results ourselves and measured them against an untouched baseline, on three graph shapes with the node count held fixed.

What we found. No run failed to locate expensive code. All eight reports name the same two regions — crossing minimization, and the network simplex serving ranking and coordinate assignment — and every binary that shipped produced byte-identical layout output.

They separate on breadth. All four Claude runs went into network simplex as well and improved three phases, measuring $1.39\times$ to $2.91\times$ end-to-end. Of the four Codex runs, one improved crossing minimization only ($1.03\times$ to $1.33\times$), and *three shipped no layout code at all*: each built a change, measured it against the workload the prompt described, found it slower or unprofitable, and reverted it. Those three binaries are the baseline recompiled, which we report as a deliberate outcome rather than a failure — and which gave us an unplanned null sample showing our per-phase noise floor is optimistic.

Repetition mattered more than we expected. Claude’s two base-prompt runs differ by $1.39\times$ against $1.67\times$ on one shape; Codex’s differ in whether anything shipped at all. A single run of an agent does not characterize it.

The parallelism prompt produced the subtlest result. Its two Claude runs land close together end-to-end, but re-measuring each at one thread shows only one is a parallel result: concurrency accounts for $2.32\times$ of one run’s $2.83\times$ on the densest shape, and for 16% of the other’s.

Which phase dominates is a property of the graph’s *shape*, not its size. At identical node count, crossing minimization ranges from 12.4% to 64.8% of layout time, setting a different ceiling on each workload — and on the sparsest one every Claude run exceeds the ceiling available to any crossing-minimization optimization at all.

Contributions.

- A controlled protocol for evaluating agent-driven optimization: balanced command ordering, which also yields an empirical noise floor from the baseline’s own spread across orderings, and a held-out evaluation harness.
- Phase-resolved measurements of eight agent runs across three graph shapes, with per-phase attribution and the shape-dependent ceiling that bounds it, plus repetitions that show how far two runs of one prompt can diverge.
- A thread-resolved decomposition of the runs that shipped concurrency, separating what their threads buy from what their serial changes buy — the measurement that decides whether a parallel-looking result is one.
- Evidence that a floor built from the baseline against itself understates cross-binary variation, from three shipped binaries that turned out to contain no change.
- A complete artifact: pinned source, every run’s diff, commit history and report, raw timing data, and the figure-generation scripts.

Section 2 covers background. Section 3 describes the protocol. Section 4 reports measurements. Sections 5–6 interpret them and state what we did not verify.

2 Background and Related Work

The dot layout pipeline. GraphViz’s dot engine draws directed graphs using the layered approach introduced by Sugiyama et al. [8] and refined by Gansner et al. [5]. It runs five phases in order, each depending on the last: *initialization* parses the file and builds the graph, *ranking* decides which layer

each node belongs to, *crossing minimization* orders nodes within a layer to reduce edge crossings, *coordinate assignment* fixes their x positions, and *spline routing* draws the edges.

Ranking and coordinate assignment are both solved with network simplex. That matters later: one agent optimized the shared code and improved both. Crossing minimization is an iterative heuristic.

Agents modifying real code. Benchmarks such as SWE-bench [6] evaluate agents on resolving real repository issues, where a test suite provides the success signal. Performance work differs in that no test can tell you whether an agent found the *right* thing to optimize, and a change that is correct but useless passes every test. Prior work on learned compiler optimization [2] targets transformations chosen in advance; here the choice of target is the object of study.

Prior work on this codebase. An earlier study by some of the present authors applied a multi-model ensemble to the same GraphViz layout code and reported a $3.78\times$ speedup from OpenMP parallelization of crossing minimization [4]. That work has not been peer-reviewed. Our measurements do not reproduce it: Table 7 bounds what any optimization of that phase can yield at 1.11 to $2.58\times$ depending on graph shape, whatever its implementation. We note also that its reported prompt template specifies the target speedup as an input requirement, which makes the reported figure difficult to interpret as an independent measurement. The present paper shares only the codebase and the general question; its protocol, measurements, and conclusions are separate.

3 Method

The design goal was that source code be the *only* thing differing between the binaries we compare. Everything below follows from that.

3.1 Sandboxes and build

Nine clones of GraphViz, all at commit 23b06521. One stays untouched as the baseline. The other eight are agent runs: two agents $\times$ two prompts $\times$ two repetitions, each on its own branch so the commit histories survive as artifacts.

The repetitions exist because a single run of an agent cannot distinguish what the agent tends to do from what it happened to do once. Every run started from the same commit with no knowledge of the others.

All nine were built with identical CMake flags (`-DCMAKE_BUILD_TYPE=Release`, Poppler and GVEdit disabled) and installed to separate prefixes. We checked that each dot loads its own build of the layout libraries rather than a shared one.

3.2 The prompts

Every run received one of two prompts. The base prompt stated the goal — make dot faster on large graphs without changing its output — and deliberately named no function, file, or phase. Finding the hot code was part of the task under study. It also:

- described the target workload shape (large layered graphs with heavy edge crossings) without supplying the evaluation graphs, so agents generated their own test inputs;
- required byte-identical `-Tdot` output and pointed at the 260 graphs in the repository’s own `tests/` directory;
- enumerated disallowed shortcuts: detecting or special-casing specific inputs, caching layout results across runs, reducing iteration counts or loosening convergence thresholds, hardcoding computed values, and modifying the test suite;
- stated that measurement would be independent, that identical builds vary by several percent on the target machine, and that a modest honest result or no result at all was an acceptable outcome.

The iteration-count prohibition matters most. Crossing minimization is iteration-bounded, so cutting its limit yields a large, real, reproducible speedup that is not an optimization. The prompt asked for that to be reported as a finding rather than shipped as a change.

The second prompt is the base prompt with one section added. It observes that dot is single-threaded while the machine’s other cores sit idle, asks the agent to treat concurrency as the primary strategy rather than an afterthought, suggests where independent work might be found (loops over ranks, components, or candidate orderings), and states that output must not depend on thread count or scheduling. It also asks for thread counts and per-thread-count speedups in the report, and for an explanation wherever the agent concluded something could not usefully be parallelized. Everything else — workload, constraints, disallowed shortcuts, deliverable — is word for word the base prompt.

We added it because no run under the base prompt introduced concurrency of any kind, in either agent. Whether that is what agents prefer or merely what they did here is exactly the sort of question a second prompt can separate.

Agents worked autonomously with no turn or time limit, committing incrementally and writing a report of what they profiled, changed, abandoned and verified. Neither had access to the evaluation harness, the benchmark graphs, or the other agent’s sandbox.

3.3 Evaluation graphs

Graph generation is deterministic. Nodes are split into equal layers and each points at `fanout` random nodes in the next layer, which is what creates the crossings layout has to resolve.

Node count is fixed at 5,000 and only the shape varies (Table 1):

Table 1: Evaluation graphs. All have 5,000 nodes. **Ranks:** $5,000 \div \text{nodes-per-rank}$. **Edges:** $(\text{ranks} - 1) \times \text{nodes-per-rank} \times \text{fanout}$.

Shape	Nodes/rank	Fanout	Ranks	Edges
sparse-deep	10	2	500	9,980
default	100	3	50	14,700
dense-shallow	200	4	25	19,200

These parameters keep every phase measurable. Wider ranks make crossing minimization so expensive that nothing else is visible next to it.

The generator does not pin ranks. Wrapping each layer in `rank=same` is legal DOT, but it supplies the answer the ranking phase exists to compute: on otherwise identical graphs that reduces ranking from 267 ms to 1 ms and removes the phase from measurement.

3.4 Measurement

All timings use `hyperfine` 1.20.0 [7] with 10 warmup and 10 timed runs. All binaries for a graph are measured in one invocation, seconds apart under the same thermal conditions; separate invocations produce order-dependent bias large enough to swamp the effects we are looking for.

`hyperfine` runs all repetitions of one command before starting the next, so a fixed order always hands the same binary the tail of any drift. We rotate the starting position and pool: four orderings, 40 timed runs per binary per graph. With nine binaries a full rotation would take nine orderings; four is enough that no binary sits at the tail of drift in more than one of them, and it keeps a single invocation under an hour.

The rotation also supplies the noise floor. The baseline is measured four times, once per ordering, so the spread among its own means is what the protocol reports for a program compared against itself.

Binaries that use threads pick their own thread count, and we measure whatever they pick rather than imposing one: that is what a user running the binary would get. The two differ — one defaults to every online core, the other to the performance cores only — so Section 4.6 measures each of them again at one thread, which is what separates a threaded gain from a serial one.

Two conventions hold throughout this paper. A *mean* is always the arithmetic mean of individual timed runs, with warmup runs excluded. A *speedup* is always the ratio of two such means — never the mean of per-run or per-ordering ratios. The two are not equal, and every ratio we report is the former.

We prefer this to benchmarking separately-built identical binaries, which we also tried. Those would be measured in their own invocation, with a different command set and a different thermal profile from the runs they are meant to bound. The baseline’s spread is measured in the same invocations, on the same graphs, against the same competitors as the results it guards.

3.5 Phase instrumentation

Phase costs are measured inside the process. An identical patch — applied by cherry-pick, so all nine diffs match byte for byte — reads a monotonic clock around each phase and writes the elapsed time to standard error. It is off unless an environment variable is set, so layout output is unchanged and the correctness check below is unaffected.

Parsing and output writing happen outside the layout routine, so the phases do not sum to wall clock. We report the difference rather than assigning it to a phase.

3.6 Correctness checking

Correctness is checked on `dot -Tdot` output, not a rendered image. The text format is deterministic, so byte-identity means something; image comparison would fold in rendering and floating-point behaviour we are not studying. We also read both diffs against the prompt’s list of disallowed shortcuts.

3.7 Platform

Experiments ran on an Apple M4 Pro (12 cores: 8 performance, 4 efficiency), 24 GB unified memory, macOS 26.5.2, with Apple Clang 17.

4 Results

4.1 Noise floor

Because command order rotates, the baseline is measured once in each position. Table 2 reports how far apart its three means land — the apparent speedup this protocol produces when nothing has changed.

Table 2: Apparent speedup measured between the baseline and itself, per graph and per phase. **Each cell:** average the baseline’s 10 timed runs within each of the four command orderings, giving four means; form all six pairwise ratios, larger divided by smaller; the cell is the largest of the six.

Measurement	sparse-deep	default	dense-shallow
End-to-end	1.034×	1.033×	1.069×
Initialization	1.022×	1.029×	1.090×
Ranking	1.041×	1.037×	1.061×
Crossing minimization	1.025×	1.052×	1.062×
Coordinate assignment	1.039×	1.078×	1.155×
Spline routing	1.055×	1.030×	1.066×

Worked example, crossing minimization on sparse-deep: the four per-ordering means are 60.96, 61.36, 61.67 and 60.18 ms, giving six pairwise ratios between 1.005 and 1.025. The largest is the cell.

The floor is not uniform, which is why we report it per graph and per phase. The noisiest cell is coordinate assignment on dense-shallow at 1.155×, and it is not a property of that phase: the fourth ordering measures 295.0 ms against 255.4, 259.5 and 260.1 in the other three, and the same ordering also inflates dense-shallow end-to-end (3176.5 ms against roughly 2975). One slow ordering raises every cell it touches. Pooling would let it set a threshold everywhere.

This floor is higher than the one we reported for a three-binary experiment on the same machine. Nine binaries take about three quarters of an hour per graph to measure, and a longer invocation gives

thermal and scheduling drift more room to separate the orderings. A floor has to be measured inside the experiment it guards, not carried over from a smaller one.

Six ratios per cell is still a small sample, so each is a rough bound. That is the cost of not conflating graphs.

Throughout the rest of this section, we treat a measured ratio as a change only if it exceeds the floor for that same phase and that same graph. Ratios that do not are marked in the tables that follow.

This floor is measured inside the same hyperfine invocations as the results it bounds, on the same graphs, against the same competing commands. The more conventional construction — benchmarking separately-built identical binaries — places them in an invocation of their own, with a different command set and thermal profile, and so describes a different experiment from the one being guarded.

4.2 Phase weights depend on shape, not size

Which phase dominates is not fixed. At identical node count, crossing minimization takes 12.4% of layout time on one shape and 64.8% on another, while coordinate assignment moves the other way, 44.3% down to 9.2%. This sets a different ceiling on each workload, and it is why one optimization gives different end-to-end results on different graphs.

Table 3 and Figure 1 give the baseline cost of each phase.

Table 3: Baseline phase cost. Phases are timed inside the process, so these are direct measurements rather than differences. **ms**: mean of that phase over all 40 timed runs (10 runs $\times$ 4 command orderings). **share**: that phase’s ms divided by the sum of the five phase ms. **Unaccounted**: wall clock minus that sum — parsing and output writing, which happen outside the layout routine.

Phase	sparse-deep		default		dense-shallow	
	ms	share	ms	share	ms	share
Initialization	114.8	23.3%	116.1	9.2%	118.8	4.1%
Ranking	75.7	15.3%	264.7	21.0%	532.5	18.4%
Crossing minimization	61.0	12.4%	571.7	45.4%	1878.5	64.8%
Coordinate assignment	218.7	44.3%	240.9	19.1%	267.5	9.2%
Spline routing	23.3	4.7%	66.3	5.3%	99.5	3.4%
Measured total	493.6	100%	1259.7	100%	2896.7	100%
Wall clock	557.5		1360.7		3029.1	
Unaccounted	64.0	11.5%	101.0	7.4%	132.4	4.4%

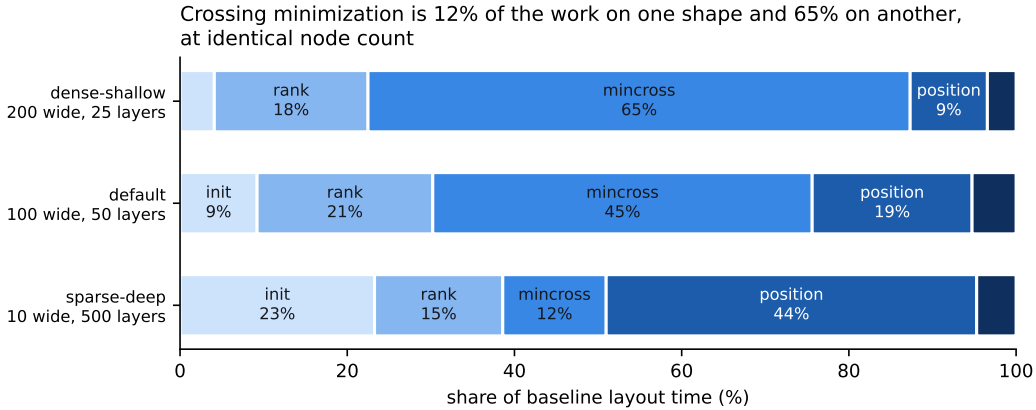


Figure 1: Baseline runtime composition by shape, at identical node count.

We report the unaccounted row rather than folding it into a phase. It is largest on sparse-deep, where the layout work is smallest and the fixed cost of parsing shows through.

4.3 End-to-end results

Table 4 and Figure 2 give layout time for all nine binaries on each shape.

Table 4: End-to-end layout time in milliseconds. **Mean and sd:** over all 40 timed runs (10 runs $\times$ 4 command orderings). **Speedup:** baseline mean divided by that binary’s mean. A dagger marks a ratio that does not exceed the end-to-end floor for that graph in Table 2; we read those as no change. **P** marks a run given the parallelism prompt.

Binary	sparse-deep		default		dense-shallow	
Baseline	557.5 $\pm$ 9.6		1360.7 $\pm$ 40.0		3029.1 $\pm$ 99.9	
Claude 1	399.9 $\pm$ 3.2	(1.39)	878.4 $\pm$ 27.7	(1.55)	1603.3 $\pm$ 64.3	(1.89)
Claude 2	334.6 $\pm$ 3.6	(1.67)	685.7 $\pm$ 20.1	(1.98)	1288.7 $\pm$ 31.9	(2.35)
Claude 1 P	349.3 $\pm$ 4.1	(1.60)	646.8 $\pm$ 26.2	(2.10)	1102.6 $\pm$ 16.1	(2.75)
Claude 2 P	378.2 $\pm$ 7.0	(1.47)	620.8 $\pm$ 30.3	(2.19)	1042.1 $\pm$ 20.7	(2.91)
Codex 1	543.1 $\pm$ 8.5	(1.03) [†]	1172.7 $\pm$ 36.5	(1.16)	2284.8 $\pm$ 62.5	(1.33)
Codex 2	559.5 $\pm$ 7.6	(1.00) [†]	1325.4 $\pm$ 49.2	(1.03) [†]	3026.6 $\pm$ 154.2	(1.00) [†]
Codex 1 P	555.8 $\pm$ 11.0	(1.00) [†]	1336.9 $\pm$ 72.0	(1.02) [†]	2974.4 $\pm$ 80.3	(1.02) [†]
Codex 2 P	554.6 $\pm$ 10.4	(1.01) [†]	1351.8 $\pm$ 108.7	(1.01) [†]	3002.7 $\pm$ 115.8	(1.01) [†]

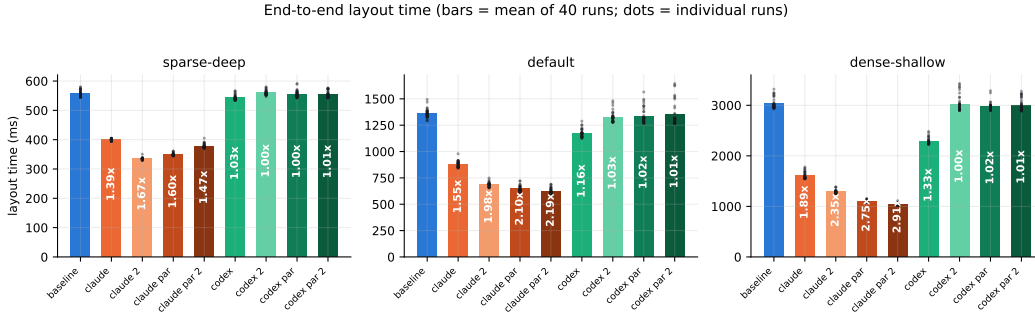


Figure 2: End-to-end layout time on all three shapes.

All four Claude runs improve all three shapes, from $1.39\times$ to $2.91\times$, and every one of those twelve ratios clears its floor by a wide margin. Among the four Codex runs, only the first shows any improvement at all: $1.16\times$ on default and $1.33\times$ on dense-shallow, with sparse-deep inside the floor. The other three Codex runs are indistinguishable from the unmodified baseline on every graph.

That is not a failure to optimize. Three of those four Codex runs shipped no layout code, by their own decision, and Section 4.5 treats them separately — both because averaging them with the first would describe a binary nobody produced, and because a binary that is the baseline recompiles tells us something useful about the floor.

The spread between repetitions of the same prompt is larger than we expected. Claude’s two base-prompt runs differ by 1.39 against 1.67 , 1.55 against 1.98 , and 1.89 against 2.35 — the second run is faster on every shape, by more than the gap between some agents. Codex’s two base-prompt runs differ more sharply still: one shipped two changes, the other shipped nothing.

4.4 Per-phase attribution

Table 5 and Figure 3 break the end-to-end result down by phase.

Thirty-nine of the 75 entries exceed their floor, and they fall in an obvious pattern: every Claude run on ranking, crossing minimization and coordinate assignment on all three graphs, plus Codex’s first

Table 5: Per-phase speedup over baseline, for the five runs that shipped layout code. **Each entry:** the baseline’s mean for that phase divided by that binary’s mean for the same phase, each mean taken over all 40 timed runs. A dagger marks an entry that does not exceed the floor for that same phase and graph in Table 2; those are no change, not small effects. **P** marks a run given the parallelism prompt.

Phase	Claude 1	Claude 2	Claude 1 P	Claude 2 P	Codex 1
<i>sparse-deep</i>					
Initialization	1.01 [†]	1.01 [†]	1.00 [†]	1.00 [†]	1.00 [†]
Ranking	2.55	2.85	4.59	4.47	0.99 [†]
Crossing minimization	1.48	1.84	2.50	1.26	1.32
Coordinate assignment	1.70	2.92	2.06	2.00	1.00 [†]
Spline routing	1.01 [†]	1.03 [†]	1.01 [†]	0.99 [†]	1.01 [†]
<i>default</i>					
Initialization	1.00 [†]	1.00 [†]	0.97 [†]	0.97 [†]	0.98 [†]
Ranking	2.07	2.53	4.36	4.19	0.99 [†]
Crossing minimization	1.91	2.67	3.28	3.85	1.53
Coordinate assignment	1.42	2.84	1.97	2.05	0.99 [†]
Spline routing	1.01 [†]	1.01 [†]	1.00 [†]	0.98 [†]	1.00 [†]
<i>dense-shallow</i>					
Initialization	1.01 [†]	1.01 [†]	1.02 [†]	1.01 [†]	1.02 [†]
Ranking	2.10	2.50	4.57	4.30	1.01 [†]
Crossing minimization	2.37	2.95	3.71	4.31	1.64
Coordinate assignment	1.28	2.89	1.98	2.03	1.02 [†]
Spline routing	1.01 [†]	1.02 [†]	1.03 [†]	1.00 [†]	1.01 [†]

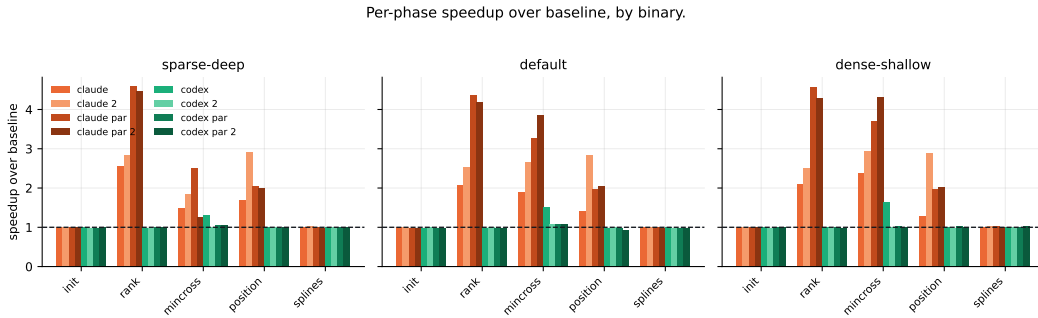


Figure 3: Per-phase speedup. Every Claude run improves ranking, crossing minimization and coordinate assignment on every shape; the one Codex run that shipped code improves crossing minimization only.

run on crossing minimization on all three. Initialization and spline routing never clear anywhere, which is the control we expect — no run touched either.

The four Claude runs divide the same three phases differently. Ranking is where the parallelism-prompt runs separate themselves: 4.19 to $4.59\times$ against 2.07 to $2.85\times$ for the base-prompt runs. Crossing minimization ranges from 1.26 to $4.31\times$, and coordinate assignment from 1.28 to $2.92\times$, with no run dominating the others on all three.

One entry is worth isolating. Claude’s second parallelism run manages $4.31\times$ on crossing minimization on dense-shallow but only $1.26\times$ on sparse-deep — the widest spread any run shows on any phase. Its concurrency is in transpose, which needs rank width to have anything to divide, and sparse-deep is ten nodes wide.

Codex’s first run improves crossing minimization by 1.32 to $1.64\times$ and nothing else. That is what produces its $1.03\times$ end-to-end on sparse-deep, inside the floor: the optimization is real on the phase, but the phase is 12.4% of that workload.

4.5 The runs that shipped nothing

Three of the four Codex runs changed no layout code at all. Their complete diff against the starting commit is the report each was asked to write, plus, in one case, an unused benchmark generator under `tools/`. Under `lib/` and `cmd/` the diff is empty. Those three binaries are the baseline, recompiled.

Each arrived there by a different route, and in each case by reverting work it had already committed:

- **Codex 2** replaced the crossing counter with a Fenwick tree, then measured a 17% regression at $25,000$ nodes — the shape the prompt named — and reverted it. Its report attributes the loss to the original scan being cache-friendly at these rank widths.
- **Codex 1 P** parallelized adjacent-rank crossing counts with OpenMP, measured $0.92\times$ at two threads, and reverted.
- **Codex 2 P** did the same with pthreads, and reverted on the grounds that the phase it could parallelize was 0.2 s against a positioning phase of 1.7 to 3.0 s.

We report these as deliberate outcomes, not as failed optimizations. Each run built something, measured it against the workload the prompt described, and concluded the change did not earn its place. The prompt explicitly allowed that outcome.

They also give us something we did not design for: three binaries that are byte-for-byte the baseline in every line of layout code, measured in the same invocations as the rest, under different names and in different command positions. Their nine end-to-end ratios run from 1.00 to $1.03\times$, which is what the floor predicts.

Their per-phase entries are less reassuring. All three measure 1.07 to $1.08\times$ on crossing minimization on sparse-deep and default — above the 1.025 and 1.052 floors for those cells — and one measures $0.93\times$ on coordinate assignment on default. Three independent binaries containing no change do not agree on a $1.07\times$ improvement by chance, so this is a position effect the baseline-against-itself construction does not capture: the floor in Table 2 compares the baseline only with copies of itself run in the same invocation, and cross-binary variation is evidently larger.

The practical consequence for this paper is small, because every result we claim clears its floor several times over. But a per-phase ratio near $1.1\times$ in this protocol should not be read as an effect, and we would not report one as such.

4.6 How much of the parallel gain is threads

The two parallelism-prompt runs from Claude both ship thread pools, and both choose their own thread count. Running them again at `GV_THREADS=1` separates what their concurrency buys from what their serial changes buy. Table 6 reports that measurement, taken in its own hyperfine invocation with two orderings and 20 timed runs per configuration, so its ratios should be read against its own baseline rather than against Table 4.

The two runs are not the same kind of result, despite landing close together in Table 4.

Table 6: Speedup over baseline at one thread and at the binary’s own default, and the ratio between them. **From threads** is the default-thread mean divided by the one-thread mean: the part of the speedup that concurrency accounts for. Baseline means are 578.2, 1368.3 and 3038.9 ms.

Binary	Threads	sparse-deep	default	dense-shallow
Claude 1 P	1	1.61×	1.99×	2.33×
Claude 1 P	default	1.64×	2.09×	2.71×
<i>from threads</i>		1.01×	1.05×	1.16×
Claude 2 P	1	1.48×	1.37×	1.22×
Claude 2 P	default	1.52×	2.18×	2.83×
<i>from threads</i>		1.03×	1.59×	2.32×

Claude’s first parallelism run is a serial optimization that also contains threads. At one thread it already reaches 1.61, 1.99 and 2.33×; enabling its threads adds 1 to 16%. Its report explains why: its concurrency sits inside network simplex behind subtree-size thresholds of 32,768 and 16,384 nodes, and our graphs have 5,000, so on this workload those paths rarely execute. What it gained, it gained from rewriting subtree relabelling as array operations.

Claude’s second parallelism run is the opposite. At one thread it is the weaker of the two on every graph. With threads it becomes the fastest binary we measured, and on dense-shallow concurrency accounts for 2.32× of its 2.83×. Its threading is in `transpose`, with a 2,000-node threshold that our graphs clear, and `transpose` is most of crossing minimization, which is 64.8% of dense-shallow.

On sparse-deep neither gains from threads (1.01 and 1.03×). Crossing minimization is 12.4% of that shape and the network-simplex subtrees are small, so on that graph both are serial results.

This is the measurement that decides how the parallelism prompt should be described. It produced two runs that both look like parallel wins end to end, and only one of them is.

4.7 Correctness

All eight binaries produce `dot -Tdot` output byte-identical to the baseline on every evaluation graph, with instrumentation both enabled and disabled: 48 comparisons, no differences.

We also ran the 260 graphs in GraphViz’s own test suite through the five binaries that shipped layout code. Every run changed `mincross.c`, which only `dot` uses; the four Claude runs also changed `ns.c`, which `neato` reaches as well, so we checked both engines. All 2,600 comparisons are identical, with matching exit codes and nothing skipped. We do not report `fdp` or `circo`: the baseline `fdp` is not reproducible against itself on these inputs, so byte comparison is meaningless for it, and baseline `circo` does not finish one test graph within an hour.

Two of the binaries create threads, so byte-identity at one thread count is not sufficient for them. We repeated the whole test-suite comparison for both at `GV_THREADS` of 1, 2 and 8 and at their own default: 2,080 further comparisons, all identical.

That check on its own would prove less than it appears to. Both binaries gate their concurrency behind input-size thresholds — 16,384- and 32,768-node subtrees in one, 2,000 nodes in the other — and almost every graph in `tests/graphs` is far below all of them, so those 2,080 comparisons largely exercise the serial paths. We therefore repeated the comparison on the three evaluation graphs, which are large enough to execute the parallel code: each binary at `GV_THREADS` of 1, 2, 4, 8, 12 and its own default, three repetitions each, for 108 comparisons. All are identical to the baseline.

Output does not depend on thread count in anything we ran, which is evidence for the agents’ determinism arguments but not a proof of race-freedom — see Section 6.

The remaining three binaries need no such check. Their layout sources are identical to the baseline’s (Section 4.5), so their output is identical by construction; they are nonetheless included in the 48 comparisons above.

Reading all eight diffs against the prompt’s disallowed shortcuts found no reduced iteration counts, early exits, input special-casing, cached layout results or modified tests. Four binaries do introduce

concurrency, which the parallelism prompt asked for and which the same prompt required to leave output unchanged; the byte-identity results above are what we have to show for that requirement.

5 Discussion

5.1 Finding the bottleneck was not the hard part

No run was told where to look, and none of the eight failed to find expensive code. Every one profiled first, and every report names the same two regions: crossing minimization, and the network simplex that serves both ranking and coordinate assignment. Several runs independently arrived at the same transformation — fusing the two symmetric crossing-count passes into one. Locating the work is evidently not what separates these runs.

Breadth is. All four Claude runs went into network simplex as well as crossing minimization, and that is where their margin comes from: ranking and coordinate assignment improve in every Claude run and in no Codex run (Table 5). Codex’s one shipping run stopped at the phase it identified first. The pattern holds across three workloads of very different composition, so it is not an inference from a single measurement.

The reports also show the two agents disagreeing about the same code rather than seeing different things. Both Codex parallelism runs argue that `transpose` cannot be parallelized without changing output, because each accepted swap changes the state the next comparison reads, and that network simplex pivots are inherently sequential. Both Claude parallelism runs parallelized exactly that code anyway — one with a speculative sweep schedule and an undo log, the other by running two independent ordering passes on private copies — and produced byte-identical output. The disagreement is not about where the time goes. It is about whether a dependency is a hard barrier or an engineering problem.

5.2 Agents optimize the workload they imagine

No run could see our evaluation graphs, so each generated its own. Every one chose graphs larger than ours: 5,000 to 25,000 nodes, against our fixed 5,000.

That choice has consequences we can measure. Claude’s first parallelism run put its concurrency behind subtree-size thresholds of 32,768 and 16,384 nodes, tuned on graphs where network simplex dominates. On our graphs those paths almost never execute, and its threads contribute between 1 and 16% (Table 6) — it scores well on the strength of a serial rewrite that happens to travel. Claude’s second parallelism run put its concurrency in `transpose` behind a 2,000-node threshold, which our graphs clear, and its threads contribute up to $2.32\times$.

Neither run was tuned to our benchmark; one simply guessed a workload closer to it. A study that reported only end-to-end numbers would have called both of them parallel wins.

5.3 Why breadth mattered: the ceiling moves with shape

Amdahl’s law [1] bounds what optimizing one phase can yield, and that bound depends on the phase’s share of runtime — which Table 3 shows is a property of the graph’s shape.

Table 7: What optimizing crossing minimization alone can achieve, against what each run measured. **Ceiling:** baseline wall clock divided by (wall clock minus that phase’s measured cost), i.e. the phase driven to zero. Remaining columns are the end-to-end speedups from Table 4, carrying the same dagger convention. Entries above the ceiling are bold.

Graph	Ceiling	Claude 1	Claude 2	Claude 1 P	Claude 2 P	Codex 1
sparse-deep	$1.12\times$	1.39	1.67	1.60	1.47	$1.03^\dagger$
default	$1.72\times$	1.55	1.98	2.10	2.19	1.16
dense-shallow	$2.63\times$	1.89	2.35	2.75	2.91	1.33

Codex’s one shipping run sits below the ceiling on all three shapes, as a run that optimized only that phase must. On sparse-deep the ceiling is $1.12\times$, so even a perfect crossing-minimization

optimization would be nearly invisible end-to-end. Its $1.03\times$ is not a failed optimization — the change works, at $1.32\times$ on the phase — but the phase is worth 12.4% of that workload.

Every Claude run *exceeds* that ceiling on sparse-deep, which is only possible by improving other phases. That is the breadth argument at its sharpest: on that workload no amount of further work on the phase every run found could have matched what they did.

The ceiling also explains why the shapes rank the runs differently. On dense-shallow, where crossing minimization is 64.8% of the work, the ceiling is $2.63\times$ and only the two parallelism runs clear it — breadth matters less there, because the one phase everybody found is most of the workload. On sparse-deep the ceiling is so low that breadth is the only thing that matters. A single benchmark graph would have hidden whichever of those two effects it did not contain.

The ceiling an optimization can reach is a property of the input, not the program, and a single benchmark graph cannot show that.

5.4 Cost

Table 8: Cost of each Claude run, as reported by its session log. All four ran `claude-opus-5`. **P** marks the parallelism prompt.

Metric	Claude 1	Claude 2	Claude 1 P	Claude 2 P
Working time	4 h 27 m	1 h 16 m	3 h 23 m	5 h 22 m
Input tokens	730	2,838	3,914	3,848
Output tokens	620,448	274,861	229,374	212,306
Cache creation	1,270,669	437,852	805,875	667,657
Cache read	89,851,473	26,712,125	36,947,373	40,939,987
Total tokens	91,743,320	27,427,676	37,986,536	41,823,798

Table 9: Cost of each Codex run, as reported by its session log. All four ran `gpt-5.6-terra`. Cached input is a subset of input; reasoning output is a subset of output. **P** marks the parallelism prompt.

Metric	Codex 1	Codex 2	Codex 1 P	Codex 2 P
Working time	23 m 42 s	12 m 55 s	9 m 50 s	9 m 30 s
Input tokens	5,114,035	2,406,206	1,684,133	1,634,219
of which cached	4,985,600	2,313,728	1,592,064	1,559,808
Output tokens	20,826	17,782	15,144	13,443
of which reasoning	4,555	4,101	5,684	3,734
Cache-write tokens	0	0	0	0
Total tokens	5,134,861	2,423,988	1,699,277	1,647,662

The gap is consistent and large. Claude’s runs took between 1 and 5 hours and between 27 and 92 million tokens; Codex’s took between 9 and 24 minutes and between 1.6 and 5.1 million. The cheapest Claude run cost roughly five times the most expensive Codex run in tokens, and eight times in wall time.

Both columns move together within each agent, and neither predicts the result. Claude’s most expensive run by tokens is its first, which is also its slowest binary of the four; its longest run by time is the one that produced the fastest binary. Codex’s two cheapest runs shipped nothing, but so did its second run, which cost twice as much.

These are not like-for-like counts. The two systems report different categories — one folds cached reads into a total that dwarfs everything else, the other reports cache reads as a subset of input — and a token is not a unit of work. We report them because they are what the logs record, and because the order-of-magnitude difference is too large to leave out, not because they support a cost-per-speedup comparison.

6 Limitations

We state these plainly because several of them bound the conclusions above.

Two runs per prompt is still a small sample. Each agent ran twice under each prompt. That is enough to show the outcome moves — Claude’s two base-prompt runs differ by more than some agent-to-agent gaps, and Codex’s differ in whether any code shipped at all — but not enough to estimate how often either outcome occurs. Two runs can show that a single run is not characteristic. They cannot tell you what is.

Our Codex result rests on one shipping run. Three of Codex’s four runs shipped no layout code, so every Codex speedup in this paper comes from a single run. We can say what that run achieved and what the other three decided; we cannot say what a typical Codex result looks like, because we have one sample of it.

Thread counts were barely explored. The main measurements give each binary whatever thread count it chooses for itself, which is what a user would get, but means the two threaded binaries are measured at different widths (all cores against performance cores only). Section 4.6 re-measures them at one thread and at their default, end to end only. We did not sweep thread counts, measure per-phase scaling, or test on a machine with a different core mix, so we cannot say where either design stops scaling or how it behaves when cores are contended.

We have not verified the absence of data races. Two of the binaries we measured create threads. Our evidence that they are safe is byte-identical output across 2,188 comparisons that vary thread count — the 260 test graphs at one, two, eight and default threads, and the three evaluation graphs at one, two, four, eight, twelve and default threads with three repetitions each — plus the safety arguments in the agents’ own reports.

That is consistent with race-freedom and is not proof of it, for three reasons. Byte-identical output is an end-state check: a race that corrupts state the layout later overwrites, or that loses to the same winner every time on this hardware, produces identical bytes. Three repetitions per configuration is a thin sample for a timing-dependent fault. And every run was on one machine with one core mix, so schedules that only occur under contention, on more cores, or on a weaker memory model were never attempted. Both Claude parallelism runs report that ThreadSanitizer could not be made to run on this machine, and we did not run one either. Running these inputs under a dynamic race detector on Linux is the check this paper is missing.

Synthetic graphs at one node count. All three shapes come from the same generator, which produces uniform rank widths and constant fanout, and all have 5,000 nodes. Real inputs to dot are irregular in both. Every agent generated its own test graphs and every one chose larger sizes than ours, up to 25,000 nodes; at least one run’s concurrency is gated behind subtree-size thresholds our graphs never reach (Section 4.6). Our numbers therefore describe these binaries on this workload, and a larger benchmark would likely rank them differently.

The noise floor understates cross-binary variation. Section 4.5 shows three binaries containing no layout change measuring up to $1.08\times$ on a phase whose floor we computed as $1.03\text{--}1.05\times$. The floor is built from the baseline against copies of itself, and evidently does not capture everything that varies between separately named commands in one invocation. Every result we claim clears its floor several times over, but a per-phase ratio near $1.1\times$ in this protocol should not be read as an effect.

One machine, one toolchain, one point in time. Every measurement is from a single Apple M4 Pro running macOS 26.5.2 and Apple Clang 17. The wins the agents found are largely memory-hierarchy wins, and their size will differ on a machine with a different cache hierarchy or core mix. The agents themselves are moving targets: these are specific model versions at specific dates, and nothing here predicts what the same prompt produces later.

Reproducibility Statement

Every number in this paper comes from data in the artifact below. No result is estimated, extrapolated, or reported from a source other than the raw measurement files.

Artifacts. Each run has its own repository. The link gives that run’s complete diff against the starting commit, together with the commit history the agent produced and the report it wrote.

- Claude, run 1:
<https://github.com/harvest7777/claude-graphviz-run/compare/main...agent-work>
- Claude, run 2:
<https://github.com/harvest7777/claude-2-graphviz-run/compare/main...agent-work>
- Claude, parallelism prompt, run 1:
<https://github.com/harvest7777/claude-parallel-graphviz-run/compare/main...agent-work>
- Claude, parallelism prompt, run 2:
<https://github.com/harvest7777/claude-parallel-2-graphviz-run/compare/main...agent-work>
- Codex, run 1:
<https://github.com/harvest7777/codex-graphviz-run/compare/main...agent-work>
- Codex, run 2:
<https://github.com/harvest7777/codex-2-graphviz-run/compare/main...agent-work>
- Codex, parallelism prompt, run 1:
<https://github.com/harvest7777/codex-parallel-graphviz-run/compare/main...agent-work>
- Codex, parallelism prompt, run 2:
<https://github.com/harvest7777/codex-parallel-2-graphviz-run/compare/main...agent-work>
- Raw timing data and session dumps:
<https://github.com/harvest7777/graphviz-results-dump>

Source. GraphViz at commit 23b06521ab916e7f4f2ae2f9ef8413a8ccc66601, from <https://gitlab.com/graphviz/graphviz.git>. Both agent branches are named agent-work and branch from that commit.

Build.

```
cmake -S <sandbox> -B <build> \  
  -DCMAKE_INSTALL_PREFIX=<prefix> \  
  -DWITH_POPPLER=OFF -DWITH_GVEDIT=OFF \  
  -DCMAKE_BUILD_TYPE=Release  
cmake --build <build> -j12 && cmake --install <build>
```

Identical for all nine sandboxes. On macOS a Bison ≥ 3.0 path must be passed explicitly, as the system Bison is too old for the GraphViz build.

Measurement. hyperfine 1.20.0, 10 warmup and 10 timed runs, all nine binaries for a given graph in a single invocation, repeated over four command orderings with a different binary leading each time. Graphs are produced by the fixed-seed generator in the artifact repository. Raw per-run timings for every configuration are in the results repository as hyperfine JSON exports, alongside the per-run phase logs.

Figures. Every figure is regenerated from the raw JSON by a single script in the artifact (`figures/make_figures.py`).

Environment. Apple M4 Pro, 12 cores (8 performance, 4 efficiency), 24 GB unified memory, macOS 26.5.2, Apple Clang 17.0.0.

Phase instrumentation. The patch that reports per-phase timings is a single commit on `lib/dotgen/dotinit.c`, applied by cherry-pick to all nine sandboxes so that the nine diffs are byte-identical. It is inactive unless `GV_PHASE_TIMING` is set in the environment.

Agents. `claude-opus-5` (Claude Code) and `gpt-5.6-terra` (Codex), each invoked four times autonomously — two runs under each of the two prompts, both reproduced in the artifact repository. No run had network access to the evaluation harness or benchmark graphs, or to any other run’s sandbox. Session logs giving each run’s wall time and token usage are in the results repository.

7 Conclusion

Eight agent runs, given identical checkouts of GraphViz and prompts naming no target, all located the same expensive code. Finding the bottleneck was evidently not the difficult part of the task.

What separated them was breadth. Every Claude run carried on into network simplex, which serves two further phases, and improves three of five on every shape we tested — $1.39\times$ to $2.91\times$ end-to-end. Codex’s one shipping run optimized the phase it identified first and measures $1.03\times$ to $1.33\times$; on the sparsest shape that optimization works, at $1.32\times$ on its phase, and still yields nothing end-to-end because the phase is 12.4% of the workload.

Codex’s other three runs shipped nothing, each after building a change, measuring it, and judging it not worth keeping. We take that at face value. It does mean a two-run study would have described whichever run it happened to draw, which is the strongest argument in this paper for repeating agent runs before characterizing an agent.

Asking for concurrency changed what the agents built but not uniformly what it bought. Two Claude runs under that prompt land close together end-to-end while only one of them owes its result to threads, and the same prompt moved Codex to prototype concurrency three times and ship it zero times.

The ceiling an optimization can reach is a property of the input, not of the program. At fixed node count, crossing minimization ranges from 12.4% to 64.8% of layout time across our three shapes, and on the sparsest one every Claude run exceeds what any crossing-minimization optimization could achieve. A single benchmark graph cannot show this.

The results are modest and we have tried to report them at the size they are. The protocol may be the more durable contribution: balance command ordering and take the floor from the baseline’s own spread across those orderings; keep the evaluation harness out of the agents’ reach; forbid the shortcuts that manufacture speedups; measure phases directly rather than by subtraction; re-measure threaded binaries at one thread before calling a result parallel. Each caught something a simpler design would have reported as a finding.

Immediate future work is to extend to a second graph size and to non-synthetic inputs, to run the concurrent code under a dynamic race detector on a platform where one works, and to complete the correctness checks named in Section 6.

References

- [1] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *AFIPS Spring Joint Computer Conference*, pages 483–485, 1967.
- [2] Chris Cummins, Volker Seeker, Dejan Grubisic, Mostafa Elhoushi, Youwei Liang, Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Kim Hazelwood, Gabriel Synnaeve, and Hugh Leather. Large language models for compiler optimization, 2023. arXiv preprint.

- [3] John Ellson, Emden Gansner, Lefteris Koutsofios, Stephen C. North, and Gordon Woodhull. Graphviz — open source graph drawing tools. In *Graph Drawing (GD 2001)*, LNCS 2265, pages 483–484, 2001.
- [4] Edwyn Estrada, Hailu Xu, and Chunhua Liao. Learning to optimize: Automated parallel code generation for Graphviz dot layout algorithms, 2025. Unpublished manuscript. OpenReview submission <https://openreview.net/forum?id=88zyE3fJzQ>.
- [5] Emden R. Gansner, Eleftherios Koutsofios, Stephen C. North, and Kiem-Phong Vo. A technique for drawing directed graphs. *IEEE Transactions on Software Engineering*, 19(3):214–230, 1993.
- [6] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [7] David Peter. hyperfine: A command-line benchmarking tool, 2023. Version 1.20.0. <https://github.com/sharkdp/hyperfine>.
- [8] Kozo Sugiyama, Shojiro Tagawa, and Mitsuhiro Toda. Methods for visual understanding of hierarchical system structures. *IEEE Transactions on Systems, Man, and Cybernetics*, 11(2):109–125, 1981.